Multi-Party Authorization Proxy

Progress Update 4 — CS 6727 Practicum

Ian Barish

AI use: deck drafted & copy-edited with Claude; research, design, and all project decisions are my own.

The Project So Far

A proxy that requires **multiple approvers** to sign off before a sensitive action runs — its headline case is publishing a package to PyPI.

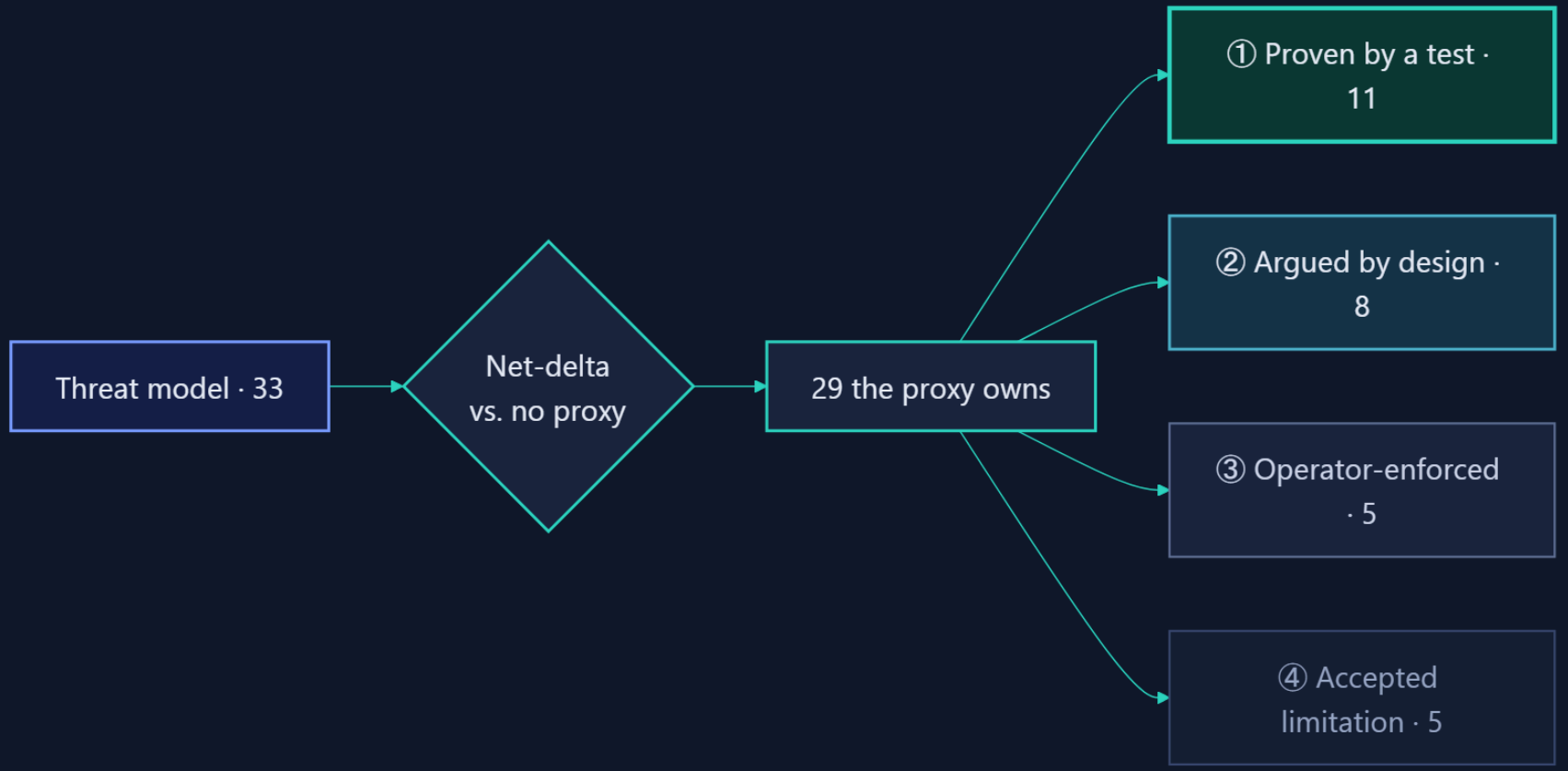
- Researched the problem: one compromised maintainer can ship to everyone
- Wrote the full design — specifications, ADRs, and a threat model
- Built the MVP — it runs end-to-end: **intercept → approve → publish**

Completed Tasks

Shifted from building the proxy to **proving it's sound** — this cycle was evaluation. Roughly 160 commits and 31 reviewed pull requests.

- **Threat model:** grew to 30+ threats, each classified and bucketed
- **Hardening:** fixed code so more threats are proven by a test
- **Evaluation suite:** runnable, built from the MVP's requirements
- **Live demo:** three acts on a real-world developer stack

How I Evaluate the Proxy



Threat 1 — A Stolen Maintainer Account

Improved · proven by 8 tests

An attacker steals a maintainer's login — the classic supply-chain entry point.

Without the proxy

That one account publishes to everyone downstream — how many recent attacks began.

With the proxy

A stolen account is only **one vote** — it can't reach quorum. The other approvers still block it.

Threat 2 — Swapping the Package After Approval

Introduced by the proxy · proven by a test

Holding the file for review opens a gap a direct upload never had.

The new gap

Swap the stored bytes after approval — ship malicious code under the quorum's sign-off.

How it's closed

Approvers sign a **SHA-256 hash**; the proxy re-checks it at publish. A swap mismatches and is refused.

Threat 3 — When the Approvers Collude

Accepted limitation · deterrence, not prevention

If a whole quorum conspires, multi-party approval is defeated by definition.

The threat

m real accounts casting m real votes *is* a valid approval — the proxy can't tell malice from consensus.

The honest limit

Can't be prevented in software. But every vote is signed and permanent — collusion is provable after the fact.

Next Steps

Final presentation

Record the video walkthrough of the working system

Final report

Start drafting — the incident case studies first

Finishing touches

Last polish on the implementation

Feedback Request

MY PLAN

I'm starting my **final report** with a **general-to-concrete** structure: open by framing multi-party approval as a **general idea** — a quorum in front of any sensitive action — then narrow to the one use case I actually built and proved: publishing a package to PyPI.

How would you frame the general part? Should I motivate it with **several example use cases**, and **how deep** do I go on the general idea vs the concrete system?